

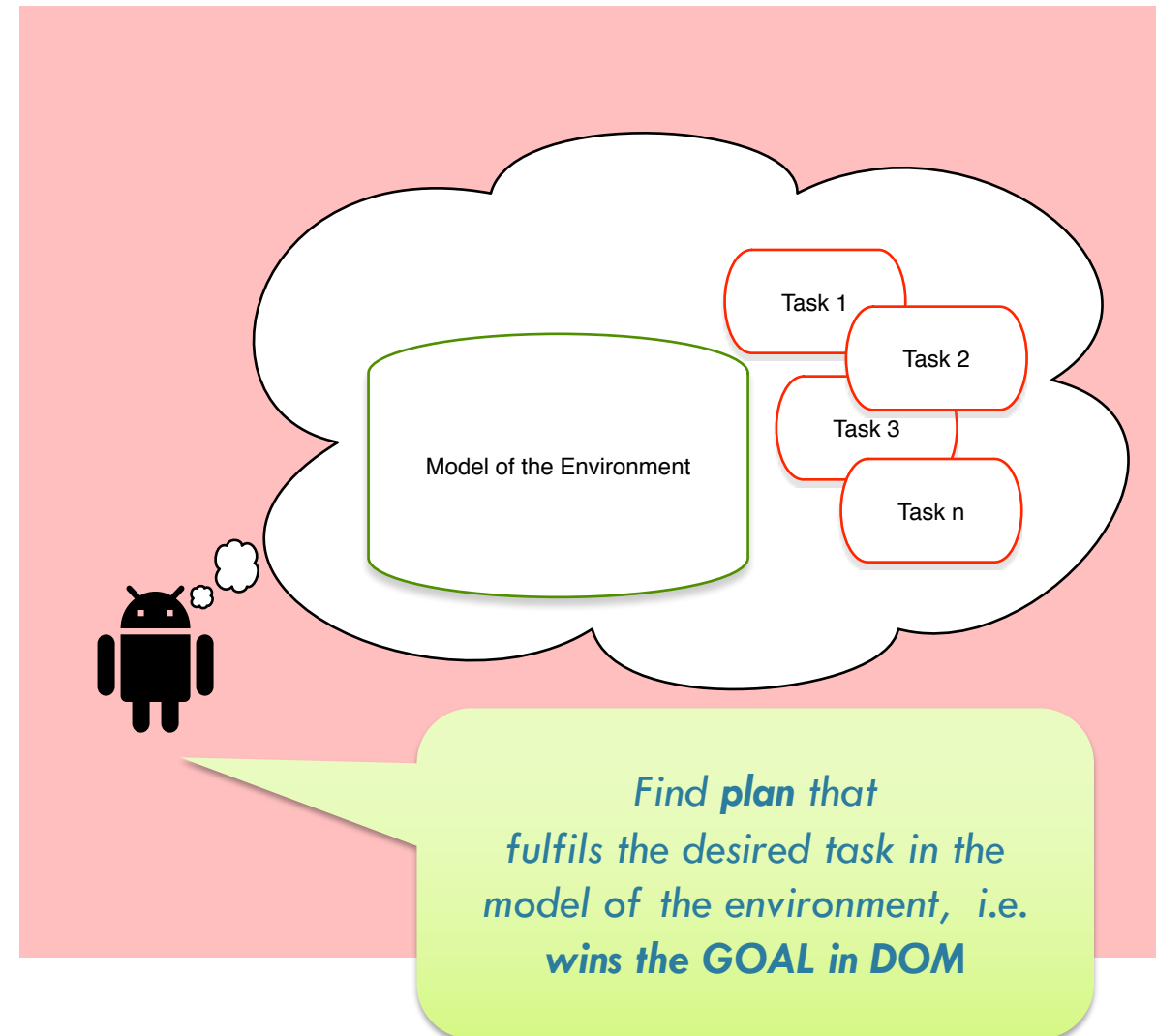
Planning by Uninformed Forward Search

Giuseppe De Giacomo

Università degli Studi di Roma “La Sapienza”

Planning in Deterministic Domains

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - STRIPS
 - ADL
 - PDDL
 - **Domain** is (or better generates) a **deterministic transition system**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan**/behavior/policy/strategy that fulfills **GOAL** in **DOM**



Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Planning in Deterministic Domains

Winning states for deterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(existential) preimage of a set $\mathcal{E}$** the following function:

$$PreE(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \delta(s, a) \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreE(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreE(W)
}
return W

```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \delta(s, a) \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in Deterministic Domains

- The kind of plan just computed is called “**Universal Plan**”
 - It is **able to say what to do to win in every single state!**
 - **If** for any reason during the execution of the plan the agent find itself in an **unexpected state**
 - i.e.: the agent expected an effect but s/he got another one

Then the **universal plan knows what to do** to win anyway
- Typically in planning it is sufficient to have a **classical plan** (not a universal one), i.e., a **sequence of actions that from the initial state leads the agent to a state satisfying the goal.**
- To compute **classical plans** there are **better algorithms** (in fact exceptionally fast) - see later in the course.

Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Uninformed Search on Directed Graphs: BFS

```
boolean BreadthFirstSearch () { //operates over a direct graph
    Queue q;
    q.enqueue(root);
    while (!q.empty()) {
        Node n = q.dequeue();
        if (!marked(n)) {
            mark(n);
            if (desired(n)) return true;
            forall (ns in successors(n))
                q.enqueue(ns);
        }
    }
    return false;
}
```

Uninformed Search on Directed Graphs: DFS

```
boolean DepthFirstSearch () { //operates over a direct graph
    Stack t;
    t.push(root);
    while (!t.empty()) {
        Node n = t.pop();
        if (!marked(n)) {
            mark(n);
            if (desired(n)) return true;
            forall (ns in successors(n))
                t.push(ns);
        }
    }
    return false;
}
```

Uninformed Search on Deterministic Domains: BFS

```
boolean BreadthFirstSearch () { //operates over a (deterministic) domain
    Queue q;
    q.enqueue(s0);
    while (!q.empty()) {
        State s = q.dequeue();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return true;
            forall (a in alpha(s))
                q.enqueue(delta(s,a));
        }
    }
    return false;
}
```

Uninformed Search on Deterministic Domains: DFS

```
boolean DepthFirstSearch () { //operates over a (deterministic) domain
    Stack t;
    t.push(s0);
    while (!t.empty()) {
        State s = t.pop();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return true;
            forall (a in alpha(s))
                t.push(delta(s,a));
        }
    }
    return false;
}
```

Uninformed Search on Deterministic Domains: BFS

Returning a Plan



```
plan BreadthFirstSearch () { //operates over a (deterministic) domain
    Queue q;
    q.enqueue( (s0,empty) );
    while (!q.empty()) {
        StatePlan (s,p) = q.dequeue();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return p;
            forall (a in alpha(s))
                q.enqueue( (delta(s,a), p+a) );
        }
    }
    return noplan;
}
```

Uninformed Search on Deterministic Domains: DFS

Returning a Plan



```
plan DepthFirstSearch () { //operates over a (deterministic) domain
    Stack t;
    t.push( (s0,empty) );
    while (!t.empty()) {
        StatePlan (s,p) = t.pop();
        if (!marked(s)) {
            mark(s);
            if (goal(s)) return p;
            forall (a in alpha(s))
                t.push( (delta(s,a), p+a) );
        }
    }
    return noplan;
}
```

Exercise: Use BFS and DFS to Return a Plan

Axiomatize the following scenario, appropriately with action precondition and effect axioms, and obtain successor state axioms.

Fluents:

- **boxClosed(s)** - The box is closed in situation s.
- **robotCloseToBox(s)** - The robot is close to the box in situation s.
- **objectInsideBox(s)** - The object is inside the box in situation s.

*Task: extract the object from the box
i.e.
Goal: not objectInsideBox*

Actions:

- **moveCloseToBox** - The robot moves close to the box. This can be done if the robot is not close to the box, and has the effect that the robot will be close to the box.
- **openBox** - The robot opens the box. This can be done if the robot is close to the box and the box is closed, and has the effect that the box will be open (that is, not closed).
- **extractObject** - The robot extracts the object inside the box. This requires that the box is open and the robot is close to the box, and has the effect that the object will be outside the box.

Initial situation description: Initially the robot is not close to the box, the box is closed, and the object is inside the box.

PDDL

```
(define (domain box)                                domain
(:requirements :strips :negative-preconditions)
(:predicates (boxClosed)
              (robotCloseToBox)
              (objInsideBox)
)

(:action moveCloseToBox
  :parameters ()
  :precondition (and (not (robotCloseToBox)))
  :effect (and (robotCloseToBox))
)

(:action openBox
  :parameters ()
  :precondition (and (robotCloseToBox) (boxClosed))
  :effect (and (not (boxClosed)))
)

(:action extractObject
  :parameters ()
  :precondition (and (not (boxClosed)) (robotCloseToBox))
  :effect (and (not (objInsideBox)))
))
```

```
(define (problem box-1)                                problem
(:domain box)
(:objects )

(:init
  ;; (not (robotCloseToBox))
  (boxClosed)
  (objInsideBox)
)

(:goal
  (not (objInsideBox))
))
```